

Книга: MicroPython за VK_RA4M2 чрез демонстрационни примери

Цел

.py VK_RA4M2

- 1.
- 2.
- 3.

Как да се ползва книгата

examples

- 1.
- 2.
- 3. .py
- 4.
- 5.

DO_WRITE DO_BOOTLOADER DO_LIGHTSLEEP
DO_DEEPSLEEP SET_DEMO_TIME

Съдържание

Част I: Първи стъпки с борда и с MicroPython

- _____
- _____

Част II: Езикови основи на MicroPython

- _____
- _____

- _____
- _____
- _____
- _____
- _____

Част III: Цифрови и аналогови входове и изходи

- _____
- _____
- _____
- _____

Част IV: Таймери, прекъсвания и време

- _____ Timer(-1)
- _____ periodic one-shot counter period output compare
input capture event count
- _____
- _____
- _____
- _____ polling Timer(-1)
- _____

Част V: Серийни интерфейси

- _____
- _____
- _____

Част VI: TouchPad, памет и системни функции

- _____
- _____
- _____ renesas.Flash /flash dataflash
- _____
- _____ machine

Част VII: Индикация и входни матрици

- _____

Част VIII: FSM като начин на мислене

- _____
- _____

- _____
- _____
- _____
- _____

Част IX: Реален пример

- _____
- _____

Част X: UART комуникация

- _____

Част XI: Външни модули по интерфейси

- _____

Част XII: asyncio като учебна секция

- _____
- _____

Част XIII: Специализирани светлинни модули

- _____
- _____ machine.WS2812

Ресурсна карта на VK_RA4M2

- 1 брой LED1 = P204
- 1 брой SW1 = P400
- USBDP = P914 USBDM = P915 USB_VBUS = P407
- 13 броя P000 P001 P002 P003 P004 P005 P006 P007 P008 P013 P014 P015 P500
- 3 броя ADC.CORE_TEMP ADC.CORE_VREF ADC.VREF
- 2 броя P014 P015
- 14 броя P107 P106 P105 P104 P113 P114 P112 P115 P608 P409 P408 P600 P304 P303
- 4 инстанции UART(0) UART(2) UART(7) UART(9)
- 2 инстанции I2C(0)=P400/P401 I2C(1)=P100/P101
- 2 инстанции I2CTarget(0)=P400/P401 I2CTarget(1)=P100/P101
- 1 канал CS=P103 SCK=P102 MISO=P100 MOSI=P101
-

-
- 12 броя P205 P206 P407 P408 P409 P410 P411 P412 P413 P414 P415 P708
- P207 = TSCAP
- 6 броя Timer(1) Timer(2) Timer(3) Timer(4) Timer(5) Timer(6)
- Timer(-1)
- 1 брой
- /flash
- renesas.Flash
- dataflash 8 KB
- SPI 1
- Timer(-1)
- SW1 P400 I2C(0).SCL
- LED1 0 1
- Pin.PULL_UP Pin.PULL_DOWN
- lightsleep deepsleep

Матрица за пълност спрямо плана

- Примерна програма `examples/20_language_basics/01_sample_program.py`
- Променливи и константи `examples/20_language_basics/02_variables_and_constants.py`
- Аритметични оператори `examples/20_language_basics/03_operators_conditions_loops.py`
- Логически оператори `examples/20_language_basics/03_operators_conditions_loops.py`
- `if` / `if-else` / `if-elif-else` `examples/20_language_basics/03_operators_conditions_loops.py`
- `goto` в MicroPython липсва
- `switch-case` в MicroPython липсва
- `while` `examples/20_language_basics/03_operators_conditions_loops.py`
- `for` `examples/20_language_basics/03_operators_conditions_loops.py`
- `do-while` еквивалент `examples/20_language_basics/03_operators_conditions_loops.py`
- Цифров изход `examples/21_digital_io/01_digital_output.py`
`examples/01_blink_async.py`
- Цифров вход `examples/21_digital_io/02_digital_input.py`

- Аналогов вход `examples/22_analog/01_analog_input_adc.py`
- Какво е ADC и как работи
`examples/22_analog/01_analog_input_adc.py`
- Измерване на ADC.read() с time.ticks_us()
`examples/23_timing/07_adc_timing_measure.py`
- PWM изход `examples/22_analog/02_pwm_output.py` `examples/03_pwm_ab_same_channel.py`
- Enums `examples/20_language_basics/04_enums.py`
- Масив `examples/20_language_basics/05_arrays.py`
- Масив от масиви `examples/20_language_basics/05_arrays.py`
- Структури `examples/20_language_basics/06_structures.py`
- Структури от структури `examples/20_language_basics/06_structures.py`
- Масив от структури `examples/20_language_basics/06_structures.py`
- Указатели като идея `examples/20_language_basics/07_pointers_equivalents.py`
- Указател към променлива/масив/структура
`examples/20_language_basics/07_pointers_equivalents.py`
- Указател към указател
- Масив от указатели
`examples/20_language_basics/08_functions.py`
- Управление на паметта в MicroPython
`examples/28_machine_misc/04_memory_management.py`
`examples/20_language_basics/07_pointers_equivalents.py`
`examples/24_i2c/04_i2ctarget_memory.py`
- gc.collect() gc.mem_free() gc.mem_alloc() micropython.mem_info()
`examples/28_machine_misc/04_memory_management.py`
- memoryview
`examples/20_language_basics/07_pointers_equivalents.py`
`examples/28_machine_misc/04_memory_management.py`
`examples/24_i2c/04_i2ctarget_memory.py`
- Функция `examples/20_language_basics/08_functions.py`
- Входни и изходни параметри `examples/20_language_basics/08_functions.py`
- Параметри чрез указател
`examples/20_language_basics/08_functions.py`
- Указател към функция
`examples/20_language_basics/08_functions.py`
- Масив от указатели към функции `operation_table`
`examples/20_language_basics/08_functions.py`
- Таймери и закъснения `examples/23_timing/01_delays_and_timers.py`
- Софтуерен таймер `examples/23_timing/04_software_timer_minus1.py`
- Хардуерен срещу софтуерен таймер `examples/23_timing/05_timer_compare_hw_sw.py`

- Хардуерен periodic и one-shot таймер
examples/23_timing/13_hardware_timer_periodic_oneshot.py
- Timer.period() Timer.counter()
examples/23_timing/13_hardware_timer_periodic_oneshot.py
- Хардуерен output compare callback
examples/23_timing/14_hardware_timer_output_compare.py
- Хардуерен input capture: период и pulse width
examples/23_timing/15_hardware_timer_input_capture.py
- Хардуерен event count и callback на всеки N импулса
examples/23_timing/16_hardware_timer_event_count.py
- time.ticks_us()
examples/23_timing/07_adc_timing_measure.py
- Прекъсвания examples/23_timing/02_interrupts.py
- Измерване на interrupt latency
examples/23_timing/08_interrupt_latency_measure.py
- Приоритети examples/30_fsm/06_priorities.py
- Забраняване/разрешаване на прекъсвания examples/23_timing/02_interrupts.py
examples/28_machine_misc/02_machine_mem_and_bootloader.py
- Четене на бутон и дебаунс examples/23_timing/03_button_debounce.py
- Четене на бутон с polling examples/23_timing/09_button_polling_events.py
- Четене на бутон по прекъсване с отложена обработка
examples/23_timing/10_button_irq_deferred.py
- Четене на два бутона с fast ASM IRQ и shared queue
examples/23_timing/17_fast_irq_two_buttons_queue.py
- Четене на бутон със софтуерен Timer(-1)
examples/23_timing/11_button_soft_timer_hold.py
- Събитие при натискане и отпускане
examples/23_timing/09_button_polling_events.py
examples/23_timing/10_button_irq_deferred.py
- Събитие при задържане examples/23_timing/09_button_polling_events.py
examples/23_timing/11_button_soft_timer_hold.py
- HOLD_START и HOLD_REPEAT examples/23_timing/11_button_soft_timer_hold.py
- Дебаунс с битово изместване examples/23_timing/12_button_shift_debounce.py
- Динамична 7-сегментна индикация
examples/29_display_scan/01_dynamic_7seg_and_keypad.py
- Прекъсвания по таймер / времево опресняване
- Матрично сканиране на бутони
examples/29_display_scan/01_dynamic_7seg_and_keypad.py
- FSM идея examples/30_fsm/01_fsm_lamp_button.py
- Графично и таблично описание на FSM
examples/30_fsm/02_fsm_table.py

- FSM с timeout `examples/30_fsm/03_fsm_timeout.py`
- Conditions vs Events `examples/30_fsm/04_conditions_vs_events.py`
- Guard функции `examples/30_fsm/05_guard_functions.py`
- Реален пример: баня `examples/31_case_study/01_bathroom_control.py`
- Декомпозиция на 2 FSM `examples/31_case_study/02_two_fsms.py`
- Комуникация междустейт машини `examples/31_case_study/02_two_fsms.py`
- Event queue `examples/30_fsm/07_event_queue.py`
- Tickless timers + sleep `examples/30_fsm/08_tickless_sleep.py`
`examples/28_machine_misc/03_sleep_modes.py`
- Работа с външни модули по интерфейси
`examples/34_external_modules/01_i2c_module_probe.py`
`examples/34_external_modules/02_spi_module_transaction.py`
`examples/34_external_modules/03_uart_module_request_response.py`
- I2C scan и безопасен probe на модул
`examples/34_external_modules/01_i2c_module_probe.py`
- SPI транзакция с CS рамка
`examples/34_external_modules/02_spi_module_transaction.py`
- UART request/response с timeout
`examples/34_external_modules/03_uart_module_request_response.py`
- NeoPixel индикация `examples/neopixel.py` `examples/neopixel_test.py`
- Захранване на малък външен RGB модул от GPIO и еднопроводен data канал
`examples/neopixel.py`
- WS2812 през специализиран C драйвер `examples/ws2812_sci.py`
`examples/ws2812_sci_test.py` `examples/ws2812_sci_p101.py`
`examples/ws2812_sci_p109.py`
- WS2812 върху валиден SCI TX/MOSI pin без външен clock pin към модула
`machine.WS2812`

Част I: Първи стъпки с борда и с MicroPython

Глава 1. Първа програма

VK_RA4M2

```
from machine import Pin # Импортираме класа Pin, чрез който управляваме GPIO пиновете.
import time # Импортираме модула time, за да правим закъснения между действията.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изходен обект за LED1 и започваме с изгасен LED.

for blink_index in range(3): # Обхождаме три последователни мигания с for цикъл.
```

```
led.value(0) # Светваме LED1 като подадем активното за платката ниво.  
time.sleep_ms(300) # Изчакаваме 300 милисекунди, за да се вижда светването.  
led.value(1) # Гасим LED1, за да завършим едно мигане.  
time.sleep_ms(300) # Изчакаваме още 300 милисекунди преди следващото мигане.
```

- Pin
- for
- sleep_ms
-

LED1

- examples/20_language_basics/01_sample_program.py

Резюме

-
- Pin sleep_ms() LED1
-

Самопроверка

1. LED1 value(0)
2. Pin.OUT sleep_ms()
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

- Pin("LED1", Pin.OUT)
-

Нататък

Глава 2. Първи асинхронни примери


```

import asyncio # Импортираме asyncio, за да правим неблокиращи задачи в MicroPython.
from machine import Pin # Импортираме класа Pin, за да управляваме LED1 като цифров изход.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изход за LED1 и започваме от изгасено състояние.

async def blink_task(): # Дефинираме асинхронна задача, която ще мига без да блокира event loop-a.
    while True: # Повтаряме мигането безкрайно.
        led.value(0) # Светваме LED1.
        await asyncio.sleep_ms(500) # Отстъпваме управлението за 500 ms.
        led.value(1) # Гасим LED1.
        await asyncio.sleep_ms(500) # Отново отстъпваме управлението за 500 ms.

async def count_seconds_task(): # Създаваме втора задача, която брой секундите.
    seconds = 0 # Началната стойност на брояча е нула секунди.
    while True: # Повтаряме и тази задача безкрайно.
        seconds += 1 # Увеличаваме брояча с една секунда.
        print("Изминали секунди:", seconds) # Показваме колко време е минало от старта
        .
        await asyncio.sleep(1) # Чакаме точно една секунда без блокиране на event loop-a.

```

- asyncio

-

- examples/01_blink_async.py
- examples/02_two_tasks.py

Резюме

-

asyncio

-

-

Самопроверка

1. await

2.

3.

Упражнения

1.

2.

3.

Ако не работи

- `await`
-

Нататък

Част II: Езикови основи на MicroPython

Глава 3. Променливи и константи

```
from micropython import const # Импортираме const, за да дефинираме истински MicroPython константи.
```

```
BOARD_NAME = "VK_RA4M2" # Това е обикновена низова променлива с името на платката.
```

```
LED_ALIAS = "LED1" # Това е още една обикновена променлива, която пази board alias на LED-a.
```

```
BLINK_COUNT = const(3) # Това е константа, която няма да променяме по време на работа.
```

```
BLINK_DELAY_MS = const(200) # Това е константа за закъснение в милисекунди.
```

```
uses_asyncio = True # Булева променлива ни показва дали платката има поддръжка на asyncio.
```

-
- `const()`
- `examples/20_language_basics/02_variables_and_constants.py`

Резюме

-

-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 4. Оператори, условия и цикли

```
left_value = 7 # Първата числова променлива участва в аритметичните операции.  
right_value = 3 # Втората числова променлива е вторият операнд.  
  
sum_value = left_value + right_value # Събираме двете стойности с оператора плюс.  
logic_result = (left_value > right_value) and (left_value != right_value) # Комбинирам  
е логически оператори.  
  
if left_value < right_value: # Проверяваме първото условие.  
    print("По-малка") # Този ред би се изпълнил, ако първото условие е вярно.  
elif left_value == right_value: # Ако първото условие е невярно, проверяваме второто.  
    print("Равна") # Този ред би се изпълнил при равенство.  
else: # Ако нито едно от горните условия не е изпълнено, влизаме в else клона.  
    print("По-голяма") # Тук попадаме с текущите примерни стойности.
```

```

for index in range(3): # Изпълняваме цикъл точно три пъти.
    print(index) # Печатаме текущата стойност на брояча.

counter = 0 # Създаваме брояч за while цикъла.
while counter < 3: # Повтаряме докато броячът е по-малък от три.
    print(counter) # Печатаме текущата стойност на брояча.
    counter += 1 # Увеличаваме брояча, за да не останем в безкраен цикъл.

attempt = 0 # Създаваме брояч за демонстрацията.
while True: # Влизаме в безкраен цикъл, който ще прекъснем с break.
    print(attempt) # Печатаме текущата стойност, като това винаги се случва поне веднъж.
    attempt += 1 # Увеличаваме брояча след изпълнението на тялото.
    if attempt >= 2: # Проверяваме условието за прекратяване след тялото на цикъла.
        break # Напускаме цикъла, което прави конструкцията еквивалент на do-while.

```

- goto
 - switch-case
 - if/elif/else break continue return
 - do-while while True break
- examples/20_language_basics/03_operators_conditions_loops.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 5. Enums в MicroPython

`from micropython import const` # Импортираме `const`, за да правим именувани константи като прост MicroPython еквивалент на `enum`.

`STATE_OFF = const(0)` # Тази константа означава състояние "изключено".

`STATE_ON = const(1)` # Тази константа означава състояние "включено".

`STATE_BLINK = const(2)` # Тази константа означава състояние "мигане".

-
-

- `examples/20_language_basics/04_enums.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 6. Масиви и таблици

```
from array import array # Импортираме array, за да покажем по-компактен числов масив от list.
```

```
adc_samples = [1200, 1215, 1198, 1222] # Това е обикновен list, който играе роля на масив от измервания.
```

```
led_table = [["LED1", "P204"], ["SW1", "P400"]] # Това е вложен масив, тоест таблица от редове и колони.
```

```
pwm_duty_values = array("H", [0, 16384, 32768, 49152, 65535]) # Тук пазим 16-битови стойности в компактен масив.
```

```
raw_bytes = bytearray([0x10, 0x20, 0x30, 0x40]) # bytearray е удобен за сурови байтове и комуникационни буфери.
```

- list
- list
- array
- bytearray

- [examples/20_language_basics/05_arrays.py](#)

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 7. Структури в MicroPython

`struct`

- `dict`
- `namedtuple`
-
-

`from collections import namedtuple # Импортираме namedtuple като лек и четим заместител на проста структура.`

`SensorRecord = namedtuple("SensorRecord", ("name", "pin", "unit")) # Описваме поле по поле една проста структура.`

`light_sensor = {"name": "Light", "adc_pin": "P000", "limits": {"dark": 800, "bright": 3000}} # dict е най-честият MicroPython заместител на структура.`

`devices = [{"name": "LED1", "pin": "P204", "kind": "output"}] # Тук правим списък от структури, както в C бихме имали масив от struct.`

- `dict`
- `dict`

- `examples/20_language_basics/06_structures.py`

Резюме

-

-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 8. Указатели и еквивалентите им в MicroPython

```
adc_list = [100, 200, 300] # Създаваме изменяем обект, който ще играе ролята на споделени данни.
```

```
alias_list = adc_list # Тук не се копира list, а и двете имена сочат към един и същ обект.
```

```
alias_list[1] = 999 # Промяната през второто име се вижда и през първото име.
```

```
tx_buffer = bytearray([1, 2, 3, 4]) # bytearray е полезен за буфери, които се редактират "на място".
```

```
tx_view = memoryview(tx_buffer) # memoryview дава прозорец към същите байтове без копиране.
```

```
tx_view[2] = 77 # Промяната през memoryview променя и оригиналния буфер.
```

-
-

- memoryview
- `machine.mem8 mem16 mem32`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 9. Функции, callbacks и таблици от функции

```
def add_values(left_value, right_value): # Дефинираме функция, която получава два вход
ни параметъра.
```

```
    return left_value + right_value # Връщаме резултата като изходна стойност.
```

```
def run_callback(callback, value): # Дефинираме функция, която приема друга функция ка
то параметър.
```

```
    callback(value) # Извикваме подадената функция с подадената стойност.
```

```
def square(value): # Това е първата функция за таблицата от функции.  
    return value * value # Връщаме квадрата на аргумента.  
  
operation_table = [square] # Това е списък от функции, тоест таблица от извикваеми обекти.
```

-
-
-
-
-
-

- `examples/20_language_basics/08_functions.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Част III: Цифрови и аналогови входове и изходи

Глава 10. Цифров изход

```
from machine import Pin # Импортираме класа Pin, за да използваме LED1 като цифров изх  
од.  
  
import time # Импортираме time, за да правим видими паузи между смените на логическото  
ниво.  
  
led = Pin("LED1", Pin.OUT, value=1) # Настройваме LED1 като изход и го оставяме изгасе  
н в началото.  
  
led.value(0) # Подаваме логическа нула на изхода.  
  
time.sleep_ms(700) # Изчакваме 700 ms, за да се види светването.  
  
led.value(1) # Подаваме логическа единица на изхода.
```

- 0
- 1
-
-

- examples/21_digital_io/01_digital_output.py
- examples/01_blink_async.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 11. Цифров вход

```
from machine import Pin # Импортираме Pin, за да четем бутона SW1 като цифров вход.  
  
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме бутона като вход с вътрешен pull-up резистор.  
  
print(button.value()) # Вземаме текущата сурова логическа стойност на входа.
```

-
-
- PULL_UP
- PULL_DOWN

- examples/21_digital_io/02_digital_input.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.

3.

Ако не работи

-
-

Нататък

Глава 12. Аналогов вход и ADC

```
from machine import ADC # Импортираме класа ADC, чрез който се четат аналоговите канал и.
```

```
temperature_adc = ADC(ADC.CORE_TEMP) # Създаваме ADC обект за вътрешния температурен канал.
```

```
vref_adc = ADC(ADC.CORE_VREF) # Създаваме ADC обект за вътрешното опорно напрежение.
```

```
print(temperature_adc.read()) # Четем суровата ADC стойност на вътрешния температурен канал.
```

```
print(vref_adc.read()) # Четем суровата ADC стойност на вътрешното опорно напрежение.
```

-
-
-
-

read_u16()

RA4M2

- ADC12 12 bit 12
/ 10 / 8 bit
- ADC(pin_or_channel,
bits=8) ADC(..., bits=10) ADC(..., bits=12)
- bits

```
from machine import ADC # Импортираме ADC за избор на резолюция при създаване на обект а.
```

```

adc_fast = ADC("P000", bits=8) # Създаваме ADC обект с 8-битова резолюция за бързо четене.
adc_balanced = ADC("P001", bits=10) # Създаваме ADC обект с 10-битова резолюция за баланс бързина/точност.
adc_precise = ADC("P002", bits=12) # Създаваме ADC обект с 12-битова резолюция за максимална точност.

print(adc_fast.read(), adc_balanced.read(), adc_precise.read()) # Четем и трите канала и показваме резултатите.

```

VK_RA4M2

- ADCLK = 50 MHz
0.31 us 8 bit 0.35 us 10 bit 0.39 us 12 bit
- ADSSTR
0.41 us 0.45 us 0.49 us
- ADC.read()
- time.ticks_us()
- ADC.read()
examples/23_timing/07_adc_timing_measure.py 8 / 10
/ 12 bit

P000 P001 P002 P003 P004 P005 P006 P007 P008 P013 P014 P015 P500

- examples/22_analog/01_analog_input_adc.py
- examples/23_timing/07_adc_timing_measure.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.

3.

Ако не работи

-
-

Нататък

Глава 13. PWM

```
from machine import Pin, PWM # Импортираме Pin и PWM, за да генерираме PWM сигнал върху валиден PWM пин.
```

```
pwm = PWM(Pin("P107"), freq=1000, duty=50) # Създаваме PWM обект с честота 1 kHz и duty 50 процента.
```

```
pwm_a = PWM(Pin("P107"), freq=1000, duty=25) # Създаваме PWM върху изход A с честота 1 kHz и duty 25 процента.
```

```
pwm_b = PWM(Pin("P106"), freq=1000, duty=75) # Създаваме PWM върху изход B със същата честота, но с duty 75 процента.
```

```
pwm_a.freq(2000) # Променяме честотата от страна на изход A, което ще промени и канала B, защото са върху един GPT канал.
```

-
-
-
-

freq()

- `examples/22_analog/02_pwm_output.py`
- `examples/03_pwm_ab_same_channel.py`

Резюме

-

-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Timer(-1)

Част IV: Таймери, прекъсвания и време

Глава 14. Закъснения, хардуерни таймери и Timer(-1)

```
import time # Импортираме time за блокиращото закъснение.
```

```
time.sleep_ms(300) # Това закъснение блокира изпълнението на останалия код.
```

```
from machine import Timer # Импортираме Timer за хардуерния таймер.
```

```
timer = Timer(1) # Използваме един от наличните хардуерни machine.Timer канали от Timer(1) до Timer(6).
```

```
timer.init(freq=4, callback=timer_callback, hard=False) # Стартираме Timer(1) на 4 Hz с callback извън hard IRQ контекст.
```

```
from machine import Pin, Timer # Импортираме Pin и Timer за демонстрацията на софтуерния таймер.
```



```

import time # Импортираме time, за да оставим таймера да работи кратко време.

led = Pin("LED1", Pin.OUT) # Вземаме LED1 като видим индикатор за сработване на callba
ck-a.

soft_timer = Timer(-1) # Създаваме софтуерен таймер чрез идентификатор -1.
soft_state = {"count": 0} # Пазим броя сработвания в mutable структура.
started = False # С този флаг ще следим дали таймерът е стартирал успешно.

def soft_timer_callback(timer_object): # Това е функцията, която ще се извиква при вся
ко сработване на софтуерния таймер.
    soft_state["count"] = soft_state["count"] + 1 # Увеличаваме брояча при всяко събит
ие.

    led.value(0 if led.value() else 1) # Превключваме LED, за да виждаме че callback-ъ
т се изпълнява.

if not started: # Ако още не сме стартирали таймера, пробваме втория стил с честота.
    try: # Влизаме в защитен блок за втория опит.
        soft_timer.init(freq=4, callback=soft_timer_callback, hard=False) # Пробваме и
нициализация с честота 4 Hz.
        started = True # Отбелязваме, че вторият стил е успял.
        print("Timer(-1) тръгна с freq API.") # Печатаме кой резервен стил е сработил.
    except Exception as second_error: # Ако и вторият стил не тръгне, само информираме
потребителя.
        print("freq API също не тръгна:", second_error) # Показваме причината и за вто
рия неуспех.

```

- Timer(1) Timer(6)
- Timer(-1)
- asyncio

time.ticks_us()

- time.ticks_us() time.ticks_diff()
- time.ticks_us() mp_hal_ticks_us()
- ADC.read()

- examples/23_timing/01_delays_and_timers.py
- examples/23_timing/04_software_timer_minus1.py
- examples/23_timing/05_timer_compare_hw_sw.py
- examples/23_timing/07_adc_timing_measure.py

- examples/23_timing/13_hardware_timer_periodic_oneshot.py
- examples/23_timing/14_hardware_timer_output_compare.py
- examples/23_timing/15_hardware_timer_input_capture.py
- examples/23_timing/16_hardware_timer_event_count.py

Подглава 14.1. Хардуерни таймери

```
Timer(1) Timer(-1)
```

```
VK_RA4M2
```

- Timer(1) Timer(6)
- Timer(-1)
- Timer.PERIODIC Timer.ONE_SHOT
- freq() period() counter()
- output compare input capture event count
- event count N
Timer.callback() period=N

```
from machine import Timer # Импортираме Timer, за да използваме хардуерните таймери.
```

```
periodic_timer = Timer(1) # Вземаме Timer(1) за периодичен режим.
```

```
oneshot_timer = Timer(2) # Вземаме Timer(2) за еднократен режим.
```

```
periodic_timer.init(mode=Timer.PERIODIC, freq=2, callback=periodic_callback, hard=False)
# Пускаме периодичен таймер на 2 Hz.
```

```
print(periodic_timer.period()) # Показваме периода в сурови AGT counts.
```

```
print(periodic_timer.counter()) # Показваме текущия суров counter.
```

```
oneshot_timer.init(mode=Timer.ONE_SHOT, freq=2, callback=oneshot_callback, hard=False)
# Пускаме еднократен таймер със същия базов период.
```

- examples/23_timing/13_hardware_timer_periodic_oneshot.py

```
from machine import Timer # Импортираме Timer, защото output compare работи през хардуерния таймер.
```

```
timer = Timer(1) # Използваме Timer(1) като базов таймер за compare събитията.
```

```
timer.init(mode=Timer.PERIODIC, freq=10, callback=cycle_callback, hard=False) # Пускаме бавен периодичен таймер на 10 Hz.
```

```

period_counts = timer.period() # Вземаме периода в сурови counts.
channel_a = timer.channel(1, mode=Timer.OC, compare=period_counts // 4, callback=compare_a_callback) # Настройваме compare A на 1/4 от периода.
channel_b = timer.channel(2, mode=Timer.OC, compare=(period_counts * 3) // 4, callback=compare_b_callback) # Настройваме compare B на 3/4 от периода.

```

- examples/23_timing/14_hardware_timer_output_compare.py

```

from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за генерация и измерване на тестов сигнал.

```

```

signal_pwm = PWM(Pin("P107"), freq=1000, duty=50) # Генерираме тестов квадратен сигнал 1 kHz на P107.

```

```

timer = Timer(1) # Използваме Timer(1) за input capture измерването.

```

```

timer.init(mode=Timer.PERIODIC, freq=1_000_000, hard=False) # Пускаме Timer(1) на 1 MHz, за да е 1 count приблизително 1 us.

```

```

period_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_PERIOD, edge=Timer.RISING) # Мерим периода на сигнала на P100.

```

- examples/23_timing/15_hardware_timer_input_capture.py

N

```

from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за броене на входни импулси.

```

```

signal_pwm = PWM(Pin("P107"), freq=200, duty=50) # Генерираме входен тестов сигнал 200 Hz на P107.

```

```

timer = Timer(1) # Използваме Timer(1) като хардуерен брояч на събития.

```

```

timer.init(mode=Timer.PERIODIC, period=20, callback=every_n_events_callback, hard=False) # Искаме callback на всеки 20 импулса.

```

```

counter_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_EVENT_COUNT, edge=Timer.RISING) # Пускаме channel(0) в event count режим.

```

```

print(counter_channel.capture()) # Четем частичния брой импулси в текущия прозорец.

```

- Timer.IC_EVENT_COUNT

- capture()

- Timer.callback()

N

- `examples/23_timing/16_hardware_timer_event_count.py`

Подглава 14.2. Практически checklist за AGT тестове

- `examples/AGT_TEST_CHECKLIST.md`

-
- `examples/        /flash/examples`
- `print()`
-

1. `examples/23_timing/13_hardware_timer_periodic_oneshot.py`
2. `examples/23_timing/14_hardware_timer_output_compare.py`
3. `P107 -> P100`
4. `examples/23_timing/15_hardware_timer_input_capture.py`
5. `P107 -> P100`
6. `examples/23_timing/16_hardware_timer_event_count.py`

- `Timer(1)`
- `Timer(2)`
- `Timer(1).counter()`
-

- Cycle-end callback count
- Compare A callback count
- Compare B callback count
-

0

- `Timer(1)` `pin=Pin("P500")` `channel(1)`

- Timer(1) pin=Pin("P501") channel(2)
- 1/4 3/4

- P107 -> P100
- 1 kHz / 50% duty Измерен период 1000
- Измерена high ширина 500
- 0

- P107 -> P100
- 200 Hz
- 20
- 1.1 s 10 11
- Частичен брой в текущия прозорец 0 19
- Приблизителен общ брой импулси 220

- Timer.IC_EVENT_COUNT
- capture()
- Timer.callback() N

- P107 -> P100
- PWM Timer
- deinit()
- periodic compare
- compare capture event count

1. examples/23_timing/13_hardware_timer_periodic_oneshot.py
2. examples/23_timing/15_hardware_timer_input_capture.py P107 -> P100
3. examples/23_timing/16_hardware_timer_event_count.py

Резюме

- Timer(-1)

-

-

Самопроверка

1. Timer(-1)
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-

-

Нататък

Глава 15. Прекъсвания

```
from machine import Pin, disable_irq, enable_irq # Импортираме Pin, disable_irq и enable_irq за демонстрацията.
```

```
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме SW1 като вход с pull-up.
```

```
def button_handler(pin_object): # Това е функцията, която ще се вика при прекъсване от бутона.
```

```
    print("IRQ") # Печатаме текст при всяко прекъсване.
```

```
button.irq(handler=button_handler, trigger=Pin.IRQ_FALLING, hard=False) # Регистрираме обработчик за падащ фронт на SW1.
```

```
irq_state = disable_irq() # Временно забраняваме прекъсванията, за да вземем консистентно копие на брояча.
```

```
snapshot = irq_state_data["count"] # Копираме брояча, докато прекъсванията са спрени.
```

```
enable_irq(irq_state) # Възстановяваме предишното състояние на прекъсванията.
```

- `time.ticks_us()`
-
-
- `examples/23_timing/08_interrupt_latency_measure.py`

- `examples/23_timing/02_interrupts.py`
- `examples/23_timing/08_interrupt_latency_measure.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 16. Дебаунс на бутон

```
if current_value != last_stable_value: # Проверяваме дали е настъпила промяна спрямо последното стабилно състояние.
```

```
    now_ms = time.ticks_ms() # Вземаме текущото време само при промяна.
```

```
    if time.ticks_diff(now_ms, last_change_ms) >= 40: # Приемаме промяната за валидна само ако е минало поне 40 ms.
```

```
        last_change_ms = now_ms # Обновяваме момента на последната приета промяна.
```

```
        last_stable_value = current_value # Приемаме новата стойност за стабилна.
```

```
    uasyncio
```

```
if stable_value == 0 and stable_count == 4: # Ако видим четири поредни проби с 0, приемаме валидно натискане.
```

```
    async_presses += 1 # Увеличаваме броя на валидните асинхронни натискания.
```

-
-

• [examples/23_timing/03_button_debounce.py](#)

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-

•

Нататък

polling Timer(-1)

Глава 17. Четене на бутон: polling, IRQ, Timer(-1), debounce и събития

VK_RA4M2

SW1

- PRESS
- RELEASE
- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

button.value()

polling

```
raw_value = button.value() # Четем суровата моментна стойност на бутона.
if raw_value != last_raw_value: # Ако видим ново сурово ниво, започваме нов debounce п
розорец.
    last_raw_value = raw_value # Запомняме новата сурова стойност.
    raw_change_ms = now_ms # Запомняме кога е започнала промяната.
if raw_value != stable_value and time.ticks_diff(now_ms, raw_change_ms) >= DEBOUNCE_MS:
    # Ако новото ниво е стабилно достатъчно дълго, приемаме ново логическо състояние.
    stable_value = raw_value # Обновяваме стабилното състояние на бутона.
    if stable_value == 0: # Ако стабилното състояние е 0, имаме валидно натискане.
        print("EVENT: PRESS") # Печатаме събитие за натискане.
    else: # Ако стабилното състояние е 1, имаме валидно отпускане.
        print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

•

•

•

polling

IRQ_FALLING | IRQ_RISING

```
def button_irq_handler(pin_object): # Това е краткият IRQ обработчик за суровите фронт  
ове на бутона.  
    irq_state_data["pending"] = 1 # Маркираме, че главният цикъл трябва да довърши лог  
ическата обработка.  
    irq_state_data["deadline_ms"] = time.ticks_add(time.ticks_ms(), DEBOUNCE_MS) # Отл  
агаме окончателното решение след debounce интервала.  
  
if pending and time.ticks_diff(now_ms, deadline_ms) >= 0: # Ако има чакаща работа и de  
bounce прозорецът е изтекъл, вземаме стабилно решение.  
    stable_candidate = button.value() # Четем реалното ниво след краткото изчакване.  
    if stable_candidate != previous_stable_value: # Ако стабилното ниво реално е ново,  
генерираме логическо събитие.  
        print("EVENT: PRESS" if stable_candidate == 0 else "EVENT: RELEASE") # Печатам  
е кое логическо събитие е настъпило.
```

-
-
-

PRESS RELEASE

Pin.irq(..., fast=True) @micropython.asm_thumb

-
-
-

P301 P302 fast ASM IRQ bytearray

```
from array import array # Импортираме array, за да държим shared 32-bit стойности в RA  
M.  
from machine import Pin # Импортираме Pin, за да вържем бутоните към external IRQ лини  
ите.  
from ctypes import addressof # Импортираме addressof, за да дадем на ASM кода адресит  
е на shared буферите.
```

```

import machine # Импортираме machine, за да ползваме disable_irq/enable_irq около cons
umer логиката.

import micropython # Импортираме micropython, защото fast IRQ handler-ите са @micropyt
hon.asm_thumb.

import time # Импортираме time, за да направим release debounce извън IRQ.

events = bytearray(16) # Това е ring buffer-ът за кратките събития от прекъсването.
event_values = array("i", [0] * 16) # Тук пазим snapshot на стойността за всяко събити
е.
state = array("i", [0, 0, 0, 0, 0, 0]) # Държим wr_idx, rd_idx, value, lost_events, pl
us_lock и minus_lock.

queue_addr = addressof(events) # Взимаме адреса на ring buffer-а за директен byte дост
ъп от ASM.

values_addr = addressof(event_values) # Взимаме адреса на snapshot буфера за директен
word достъп от ASM.

state_addr = addressof(state) # Взимаме адреса на state блока за директен word достъп
от ASM.

exec(build_button_isr_source("plus_isr", +1, 1), globals(), globals()) # Генерираме fa
st ASM ISR за бутона "+".

exec(build_button_isr_source("minus_isr", -1, 2), globals(), globals()) # Генерираме f
ast ASM ISR за бутона "-".

plus_button = Pin("P301", Pin.IN, Pin.PULL_UP) # Настройваме P301 като вход с pull-up
за бутона "+".

minus_button = Pin("P302", Pin.IN, Pin.PULL_UP) # Настройваме P302 като вход с pull-up
за бутона "-".

plus_button.irq(trigger=Pin.IRQ_FALLING, handler=plus_isr, fast=True) # Връзваме fast
IRQ на падащ фронт за бутона "+".

minus_button.irq(trigger=Pin.IRQ_FALLING, handler=minus_isr, fast=True) # Връзваме fas
t IRQ на падащ фронт за бутона "-".

while True: # Главният цикъл остава извън IRQ и само довършва debounce и печата събити
ята.
    handle_release_debounce() # Re-arm-ваме бутона едва след стабилно отпускане.

    irq_state = machine.disable_irq() # Кратко спираме IRQ, за да вземем консистентен
snapshot на опашката.

    event_code, event_value, lost_events = pop_event() # Вземаме едно събитие и snapsh
ot стойността му от shared буфера.

    machine.enable_irq(irq_state) # Връщаме IRQ веднага след кратката критична секция.

    if event_code == 1: # Ако кодът е 1, печатаме събитие за бутона "+".
        print("PLUS value =", event_value) # Показваме snapshot стойността за натиска
не на бутона "+".

    elif event_code == 2: # Ако кодът е 2, печатаме събитие за бутона "-".
        print("MINUS value =", event_value) # Показваме snapshot стойността за натиска
не на бутона "-".

```

```

    if lost_events: # Ако ring buffer-ът е бил пълен, показваме, че има изпуснати съби-
тия.

        print("Queue overflow, lost events =", lost_events) # Печатаме диагностиката з
а overflow извън IRQ.

        time.sleep_ms(2) # Оставяме кратък sleep, за да е четим изходът и да работи releas
е debounce логиката.

```

-
-
-
-

```

                                build_button_isr_source(...)

@micropython.asm_thumb

```

- examples/23_timing/17_fast_irq_two_buttons_queue.py

```

    Timer(-1)

```

```

    Timer(-1)

```

```

soft_timer.init(period=SAMPLE_MS, mode=Timer.PERIODIC, callback=button_sample_callback)
# Стартираме Timer(-1) като периодичен sampler на бутона.

if timer_state["history"] == 0x00 and timer_state["stable_value"] != 0: # Ако последни
те осем проби са 0, приемаме стабилно натискане.

    timer_state["event_code"] = EVENT_PRESS # Записваме PRESS събитие за основния цикъ
л.

elif timer_state["stable_value"] == 0 and not timer_state["hold_started"] and time.tick
s_diff(now_ms, timer_state["press_start_ms"]) >= LONG_PRESS_MS: # Ако бутонът е натисн
ат достатъчно дълго, излъчваме начало на задържане.

    timer_state["event_code"] = EVENT_HOLD_START # Записваме HOLD_START събитие.

elif timer_state["stable_value"] == 0 and timer_state["hold_started"] and time.ticks_di
ff(now_ms, timer_state["next_repeat_ms"]) >= 0: # Ако вече сме в задържане и е дошло в
реме за повторение, излъчваме repeat.

    timer_state["event_code"] = EVENT_HOLD_REPEAT # Записваме HOLD_REPEAT събитие.

```

- PRESS 1 -> 0
- RELEASE 0 -> 1

- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

N

```
history = ((history << 1) | button.value()) & 0xFF # Премества историята наляво и до
бавяме новата проба като най-млад бит.
```

```
if history == 0x00 and stable_value != 0: # Ако последните осем проби са натиснати, пр
иемаме стабилно PRESS събитие.
```

```
    print("EVENT: PRESS") # Печатаме събитие за натискане.
```

```
elif history == 0xFF and stable_value != 1: # Ако последните осем проби са отпуснати,
приемаме стабилно RELEASE събитие.
```

```
    print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

history

- 0x00
- 0xFF
-

• polling

•

• Timer(-1)

•

- examples/23_timing/09_button_polling_events.py
- examples/23_timing/10_button_irq_deferred.py
- examples/23_timing/17_fast_irq_two_buttons_queue.py
- examples/23_timing/11_button_soft_timer_hold.py
- examples/23_timing/12_button_shift_debounce.py
- examples/23_timing/02_interrupts.py
- examples/23_timing/03_button_debounce.py
- examples/23_timing/04_software_timer_minus1.py

Резюме

-
- polling fast ASM IRQ Timer(-1)
- PRESS RELEASE HOLD

Самопроверка

1. PRESS
2. Timer(-1) polling
3. fast ASM IRQ
4. 0x00 0xFF

Упражнения

1. 09_button_polling_events.py LONG_PRESS CLICK
2. CLICK 10_button_irq_deferred.py
PRESS RELEASE
3. 17_fast_irq_two_buttons_queue.py P301 P302
4. 12_button_shift_debounce.py

Ако не работи

- PRESS RELEASE
- LONG_PRESS HOLD_REPEAT

Нататък

Глава 18. RTC

```
from machine import RTC # Импортираме RTC, за да работим с часовника в реално време.  
  
rtc = RTC() # Създаваме обект за достъп до вградения RTC модул.  
print(rtc.info()) # Печатаме диагностичната информация за стартиране на RTC.  
print(rtc.calibration()) # Печатаме текущата калибрационна стойност.  
print(rtc.datetime()) # Четем текущата дата и час като осемелементен tuple.
```

-
-
-

- `examples/23_timing/06_rtc_basic.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Част V: Сериен интерфейс

Глава 19. I2C master и runtime избор на пинове

```
from machine import I2C # Импортираме I2C за работа като master върху наличния хардуерен контролер.
```

```
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на 100 kHz върху бордовите пинове P100 и P101.
```

```
print(i2c.scan()) # Сканираме шината за устройства с 7-битов адрес.
```

```
from machine import I2C, Pin # Импортираме I2C и Pin, за да покажем runtime избиране на SCL и SDA.
```

```
i2c = I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2C(1) на 400 kHz с изрично зададени пинове.
```

- I2C(0) P400/P401
- P400 SW1

- examples/24_i2c/01_i2c_master_basic.py
- examples/24_i2c/02_i2c_runtime_pins.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 20. SoftI2C и I2CTarget

```
from machine import SoftI2C, Pin # Импортираме SoftI2C и Pin за софтуерна I2C шина върху GPIO.
```

```
soft_i2c = SoftI2C(scl=Pin("P105", Pin.OPEN_DRAIN), sda=Pin("P104", Pin.OPEN_DRAIN), freq=100000) # Създаваме SoftI2C на 100 kHz.
```

```
from machine import I2CTarget, Pin # Импортираме I2CTarget и Pin за работа в target и slave режим.
```

```
memory_buffer = bytearray(range(16)) # Подготвяме малък memory-backed буфер, който target-ът ще обслужва.
```

```
target = I2CTarget(1, addr=0x42, mem=memory_buffer, mem_addrsize=8, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2CTarget(1) върху стандартните пинове на канал 1.
```

-
-

- `examples/24_i2c/03_softi2c_basic.py`
- `examples/24_i2c/04_i2ctarget_memory.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.

3.

Ако не работи

-
-

Нататък

Глава 21. Hardware SPI и SoftSPI

```
from machine import SPI, Pin # Импортираме SPI и Pin за достъп до хардуерния SPI канал
```

.

```
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P100"), cs=Pin("P103")) # Създаваме SPI обект върху валидните board пинове.
```

```
from machine import SoftSPI, Pin # Импортираме SoftSPI и Pin за софтуерен SPI върху GPIO.
```

```
soft_spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=Pin("P105"), mosi=Pin("P104"), miso=Pin("P106")) # Създаваме софтуерен SPI обект върху три GPIO пина.
```

SPI 1

- `examples/25_spi/01_spi_basic.py`
- `examples/25_spi/02_softspi_basic.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.

3.

Упражнения

1.

2.

3.

Ако не работи

-
-

Нататък

Част VI: TouchPad, памет и системни функции

Глава 22. TouchPad: базово четене

```
from machine import Pin, TouchPad # Импортираме Pin и TouchPad за работа с CTSU вход.  
  
touch = TouchPad(Pin("P205")) # Създаваме TouchPad върху валиден touch пин P205.  
touch.config(500) # Задаваме праг 500 за метода value().  
print(touch.read()) # Четем необработената CTSU стойност.  
print(touch.value()) # Преобразуваме стойността към 0 или 1 според прага.  
print(touch.read_value()) # Взимаме едновременно count, value и последна FSP грешка.
```

-
-
-

- `examples/26_touchpad/01_touchpad_basic.py`

Резюме

-

-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 23. TouchPad: диагностика, cached API и асинхронно следене

```
TouchPad.sample_rate(20) # Стартираме глобалния cooperative sampler на 20 пълни scan-a
                           в секунда.
touch.start() # Стартираме non-blocking scan, ако не е в ход такъв.
TouchPad.service() # Придвижваме вътрешния sampler една стъпка от VM контекст.
print(touch.ready(), touch.read_cached(), touch.value_cached()) # Показваме cached със-
тоянието.
print(touch.diagnose(8)) # Печатаме резултат от кратък диагностичен цикъл.
```

```
TouchPad.sample_rate(50) # Стартираме sampler на 50 scan-a в секунда.

async def touch_task(): # Асинхронна задача за следене на TouchPad.
    while True: # Безкрайно следене.
        if tp.ready(): # Проверяваме дали пробата е готова.
```

```
        print(tp.read_cached(), tp.value_cached(), tp.age_ms()) # Печатаме cached
стойностите.
    await asyncio.sleep_ms(20) # Сканираме на 20 ms.
```

- examples/26_touchpad/02_touchpad_diagnostics.py
- examples/04_touchpad_async.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

renesas.Flash /flash dataflash

Глава 24. renesas.Flash, /flash и dataflash

renesas.Flash

```
import os # Импортираме os, за да покажем съдържанието на файловата система /flash.
import renesas # Импортираме renesas модула, който излага Flash block device.

flash = renesas.Flash() # Създаваме block device обект върху вътрешната flash памет.
sector = bytearray(16) # Подготвяме малък буфер за четене на първите байтове от блок 0
.
flash.readblocks(0, sector) # Четем първите 16 байта от блок 0 в буфера.
print(os.listdir("/flash")) # Показваме кои файлове има във вътрешната файлова система
.
```

dataflash

```
import dataflash # Импортираме dataflash модула за достъп до отделната Data Flash обла
ст.

print(dataflash.size()) # Печатаме общия размер на Data Flash областта.
print(dataflash.block_size()) # Печатаме размера на erase блока.
print(dataflash.write_size()) # Печатаме минималния размер за запис.
print(dataflash.read(0, 16)) # Четем първите 16 байта от Data Flash.
```

dataflash

LED1

```
from machine import Pin # Импортираме Pin за управление на LED1.
import dataflash # Импортираме dataflash за устойчиво съхранение на състояние.

led = Pin("LED1", Pin.OUT, value=1) # Използваме board alias-а LED1, започваме изгасен
.
state = dataflash.read(0, 1)[0] # Връщаме стойността на първия байт.

if state == 0x00: # Ако предишният старт е записал 0x00, сега правим erase към 0xFF.
    dataflash.erase_block(0) # Изтриваме блок 0, всички байтове стават 0xFF.
    blinks_per_sec = 2 # Бавно мигане означава erase.
else: # Иначе приемаме, че байтът е 0xFF и записваме 0x00.
    dataflash.write(0, bytes([0x00])) # Записваме 0x00 в първия байт.
    blinks_per_sec = 10 # Бързо мигане означава write.
```

dataflash

VK_RA4M2

LED1

P204

- 0xFF erase_block(0)
- 0x00 write(0, bytes([0x00]))

-
-
-

VK_RA4M2

- 384 KB 128 KB
- 290 KB 94 KB
FLASH_FS /flash
- 8 KB dataflash
- /flash FAT 512 bytes
- 94 KB

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

- renesas.Flash /flash dataflash
-
-

Самопроверка

1. renesas.Flash /flash dataflash
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-

-

Нататък

Глава 25. Управление на паметта в MicroPython

```
import gc # Импортираме gc за контрол и измерване на garbage-collected heap-a.
import micropython # Импортираме micropython за mem_info и други runtime helper-и.

gc.collect() # Пускаме GC в предвидим момент, за да започнем от по-чисто състояние.
print(gc.mem_free(), gc.mem_alloc()) # Печатаме свободния и заетия heap.
micropython.mem_info() # Печатаме обобщена информация за heap-a.
```

```
buffer = bytearray(32) # Създаваме един буфер, който ще използваме.
window = memoryview(buffer)[8:12] # Вземаме прозорец към част от буфера без копиране.
window[0] = 123 # Променяме първия байт през memoryview.
print(list(buffer[8:12])) # Показваме, че оригиналният буфер е променен на място.
```

-

-

-

-

VK_RA4M2

-

_heap_end

_heap_start

-

8 KB

- /flash dataflash

- gc.collect()
- gc.mem_free() gc.mem_alloc()
- gc.threshold()
- micropython.mem_info() micropython.mem_info(1)

- `micropython.heap_lock()` `micropython.heap_unlock()`

- bytearray

- `memoryview`

- `gc.collect()`

-

- `machine.mem8/mem16/mem32`

- 07_pointers_equivalents.py

- 04_i2ctarget_memory.py

- 01_flash_blockdev.py 02_dataflash_basic.py

- `examples/28_machine_misc/04_memory_management.py`

- `examples/20_language_basics/07_pointers_equivalents.py`

- `examples/24_i2c/04_i2ctarget_memory.py`

- `examples/27_storage/01_flash_blockdev.py`

- `examples/27_storage/02_dataflash_basic.py`

Резюме

-

-

-

Самопроверка

- 1.

- 2.

- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

machine

Глава 26. machine модулът и порт-специфично поведение

```
import machine # Импортираме machine, за да покажем системните функции на порта.

uid = machine.unique_id() # Четем уникалния 128-битов идентификатор на MCU.
uid_hex = "".join("{:02x}".format(byte_value) for byte_value in uid) # Преобразуваме 6
айтовете до четим hex низ.

print("machine.freq() =", machine.freq()) # Показваме текущата честота на ядрото.
print("machine.unique_id() =", uid_hex) # Печатаме уникалния идентификатор в hex вид.
print("machine.reset_cause() =", machine.reset_cause()) # Печатаме причината за послед
ния reset.

machine.info() # Печатаме подробна системна информация от самия порт.


irq_state = machine.disable_irq() # Временно забраняваме прекъсванията и пазим предишн
ото състояние.

machine.enable_irq(irq_state) # Възстановяваме прекъсванията веднага след демонстрация
та.

print(hasattr(machine, "mem32")) # Показваме дали портът излага 32-битов memory access
.


DO_LIGHTSLEEP = False # Оставяме lightsleep изключен по подразбиране.
DO_DEEPSLEEP = False # Оставяме deepsleep изключен по подразбиране.
```

```

print("Извикваме кратък machine.lightsleep(20).") # Подготвяме потребителя за кратък s
leep.
machine.lightsleep(20) # Извикваме кратък lightsleep за 20 ms.

if DO_LIGHTSLEEP: # Само ако е включено, пробваме по-дълъг lightsleep.
    machine.lightsleep(100) # Извикваме lightsleep за 100 ms.

```

- machine.info
- machine.freq
- machine.unique_id
- machine.reset_cause
- disable_irq enable_irq
- mem8 mem16 mem32
- machine.bootloader
- sleep lightsleep deepsleep

- machine.freq()

100 MHz	50 MHz
---------	--------
-
- | | |
|--------|---------|
| | 15.4 mA |
| 6.1 mA | 4.4 mA |
- | | |
|-------------|--------|
| | 0.7 uA |
| 5 ... 16 uA | |
- | | |
|--------|--------|
| 12-bit | 0.8 mA |
|--------|--------|

- examples/28_machine_misc/01_machine_info_and_id.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

- machine
-
-

Самопроверка

1. machine

2.

3.

Упражнения

1.

2.

3.

Ако не работи

-
-

Нататък

Част VII: Индикация и входни матрици

Глава 27. Динамична 7-сегментна индикация и матрично сканиране

```
from machine import Pin # Импортираме Pin, за да покажем базовото сканиране на външен дисплей и бутони.
```

```
import time # Импортираме time за краткото динамично опресняване.
```

```
segment_pins = [Pin(name, Pin.OUT) for name in ("P105", "P104", "P113", "P114", "P115", "P304", "P303")] # Това са седемте сегмента а..g.
```

```
digit_pins = [Pin(name, Pin.OUT) for name in ("P608", "P409")] # Това са двата общи извода за две цифри.
```

```
patterns = {"1": (0, 1, 1, 0, 0, 0, 0), "2": (1, 1, 0, 1, 1, 0, 1)} # Това са сегментните шаблони за две примерни цифри.
```

```
for refresh_step in range(10): # Повтаряме кратък опреснителен цикъл десет пъти.
```

```
    for digit_index, symbol in enumerate(("1", "2")): # Показваме символ 1 на първата цифра и 2 на втората цифра.
```

```
        for digit_pin in digit_pins: # Първо изключваме всички цифри, за да няма ghosting.
```

```
            digit_pin.off() # Изключваме текущата цифра.
```

```

        for segment_pin, level in zip(segment_pins, patterns[symbol]): # Зареждаме нуж
ния сегментен шаблон за текущия символ.

            segment_pin.value(level) # Настройваме сегмента според шаблона.

        digit_pins[digit_index].on() # Включваме само текущата цифра за кратко време.

        time.sleep_ms(5) # Държим цифрата включена кратко, за да се получи динамична и
ндикация.

row_pins = [Pin(name, Pin.OUT) for name in ("P408", "P600")] # Това са редовете на при
мерна 2x2 клавиатурна матрица.

col_pins = [Pin(name, Pin.IN, Pin.PULL_UP) for name in ("P301", "P302")] # Това са кол
оните на примерната 2x2 клавиатурна матрица.

pressed_keys = [] # Подготвяме списък за натиснати клавиши от матрицата.

for row_index, row_pin in enumerate(row_pins): # Сканираме всеки ред поотделно.
    for reset_row in row_pins: # Първо изключваме всички редове.
        reset_row.off() # Изключваме реда.
    row_pin.on() # Активираме текущия ред.
    for col_index, col_pin in enumerate(col_pins): # Проверяваме всяка колона в активн
ия ред.
        if col_pin.value() == 0: # Ако колоната падне на 0, приемаме че бутонът е нати
снат.
            pressed_keys.append((row_index, col_index)) # Записваме координатите на на
тиснатия бутон.

```

- [examples/29_display_scan/01_dynamic_7seg_and_keypad.py](#)

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Част VIII: FSM като начин на мислене

Глава 28. Минимална FSM и таблична FSM

```
STATE_OFF = "OFF" # Това е първото състояние на лампата.
STATE_ON = "ON" # Това е второто състояние на лампата.
EVENT_BUTTON = "BUTTON" # Това е събитието от бутона.

state = STATE_OFF # Започваме с изключена лампа.

def handle_event(current_state, event_name): # Дефинираме преходната функция на машина
та на състояния.

    if current_state == STATE_OFF and event_name == EVENT_BUTTON: # Ако сме в OFF и бу
тонът е натиснат, минаваме в ON.

        return STATE_ON # Връщаме новото състояние ON.

    if current_state == STATE_ON and event_name == EVENT_BUTTON: # Ако сме в ON и буто
нът е натиснат, минаваме в OFF.

        return STATE_OFF # Връщаме новото състояние OFF.

    return current_state # Ако няма преход, оставаме в същото състояние.


state = "OFF" # Започваме в състояние OFF.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "BUTTON"): "OFF"} # В таблицата оп
исваме преходите като двойка от state и event.

event_sequence = ["BUTTON", "BUTTON", "BUTTON", "BUTTON"] # Симулираме поредица от съб
ития.
```

OFF --BUTTON--> ON
ON --BUTTON--> OFF

- examples/30_fsm/01_fsm_lamp_button.py
- examples/30_fsm/02_fsm_table.py

Резюме

-
-
-

Самопроверка

- 1.
2. if
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 29. FSM с timeout

```
state = "OFF" # Започваме с изключена лампа.  
  
transition_table = {("OFF", "BUTTON"): "ON", ("ON", "TIMEOUT"): "OFF", ("ON", "BUTTON")  
: "OFF"} # Описваме преходите с timeout.
```

```
event_sequence = ["BUTTON", "TIMEOUT", "BUTTON", "BUTTON"] # Симулираме бутон и timeout събития.
```

- `examples/30_fsm/03_fsm_timeout.py`

Резюме

-
-
-

Самопроверка

- 1.
2. `if`
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 30. Conditions vs Events

```
light_level = 600 # Това е моментното ниво от светлинния сензор в симулацията.
```

```
dark_threshold = 800 # Това е прагът, под който приемаме че е тъмно.
```

```
event_name = "BUTTON" # Това е дискретното събитие от бутон.
```

```
condition_is_dark = light_level < dark_threshold # Това е condition, която може да е вярна дълго време.
```



```
if condition_is_dark and event_name == "BUTTON": # Комбиниране условие и събитие в общо решение.  
    print("Лампата може да се включи, защото е тъмно и има бутонно събитие.") # Печатаме положителния изход.  
else: # Ако комбинацията не е изпълнена, не включваме лампата.  
    print("Няма достатъчно условия за включване.") # Печатаме отрицателния изход.
```

-
-

• [examples/30_fsm/04_conditions_vs_events.py](#)

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

if

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 31. Guard функции

```

context = {"light": 500, "pir": True} # Подготвяме контекст със стойности от сензори.
state = "OFF" # Започваме в състояние OFF.
event_name = "MOTION" # Симулираме събитие за засечено движение.

def is_dark(data): # Това е първата guard функция.
    return data["light"] < 800 # Връщаме True само ако е достатъчно тъмно.

def has_motion(data): # Това е втората guard функция.
    return data["pir"] is True # Връщаме True само ако PIR сензорът вижда движение.

if state == "OFF" and event_name == "MOTION" and is_dark(context) and has_motion(context): # Комбинираме event, state и guard функции.
    state = "ON" # Минаваме в ON само ако всички проверки са успешни.

```

- [examples/30_fsm/05_guard_functions.py](#)

Резюме

-
-
-

Самопроверка

- 1.
2. `if`
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 32. Приоритети и event queue

```
pending_events = ["BUTTON", "ERROR", "TIMEOUT"] # Симулираме едновременно чакащи събития.
```

```
priority_order = {"ERROR": 0, "BUTTON": 1, "TIMEOUT": 2} # По-малкото число означава по-висок приоритет.
```

```
pending_events.sort(key=lambda event_name: priority_order[event_name]) # Подреждаме събитията по приоритет.
```

```
event_queue = [] # Създаваме проста event queue като обикновен Python list.
```

```
event_queue.append(("BUTTON", {"source": "Sw1"})) # Добавяме събитие от бутон.
```

```
event_queue.append(("PIR", {"source": "pir_sensor"})) # Добавяме събитие от PIR сензор.
```

```
event_queue.append(("TIMEOUT", {"ms": 30000})) # Добавяме timeout събитие.
```

```
while event_queue: # Обработваме опашката, докато има чакащи събития.
```

```
    event_name, payload = event_queue.pop(0) # Взимаме най-старото събитие от началото на опашката.
```

```
    print("Обработваме", event_name, "с payload", payload) # Печатаме кое събитие се обработва сега.
```

-
-
-
-

- `examples/30_fsm/06_priorities.py`

- `examples/30_fsm/07_event_queue.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

if

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Глава 33. Tickless timers и sleep идея

```
import machine # Импортираме machine за кратък lightsleep между събитията.
import time    # Импортираме time за работа с ticks_ms и ticks_diff.

tasks = [("lamp_timeout", 120), ("fan_timeout", 450), ("display_refresh", 40)] # Подготвяме три задачи с различни крайни срокове в милисекунди.
start_ms = time.ticks_ms() # Запомняме началния момент.
next_deadline_ms = min(deadline for _, deadline in tasks) # Избираме най-близкия срок като tickless цел.
sleep_ms = max(0, next_deadline_ms - time.ticks_diff(time.ticks_ms(), start_ms)) # Изчисляваме колко време може да се спи до следващото събитие.
machine.lightsleep(sleep_ms) # Спим точно до най-близкото планирано събитие с lightsleep.
elapsed_ms = time.ticks_diff(time.ticks_ms(), start_ms) # Изчисляваме изминалото време след съня.
ready_tasks = [task_name for task_name, deadline in tasks if deadline <= elapsed_ms] # Избираме всички задачи, които вече са изтекли.
```

-

-
-

- examples/30_fsm/08_tickless_sleep.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

-
-
-

Самопроверка

- 1.
2. `if`
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Част IX: Реален пример

Глава 34. Управление на осветление и вентилация за баня

```
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

led = Pin("LED1", Pin.OUT, value=1) # LED1 = P204, active-low: value=1 означава изгасе
н.
```

```

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, active-low: value=0 означава на
тиснат.

sensors = {"pir": True, "door_open": False, "light_level": 300, "humidity": 82} # Прим
ерни входни данни от четири сензора.

outputs = {"lamp": False, "fan": False} # Два управлявани изхода за лампа и вентилатор
.

button_pressed = (button.value() == 0) # SW1 е active-low: 0 означава натиснат.

if sensors["pir"] and sensors["light_level"] < 800: # Ако има движение и е тъмно, вклю
чваме лампата.
    outputs["lamp"] = True # Включваме лампата.

if button_pressed: # Ако бутонът е натиснат, също включваме лампата.
    outputs["lamp"] = True # Ръчно включване от потребителя.

if sensors["humidity"] > 75 or sensors["door_open"]: # Ако влажността е висока или вра
тата е отворена, включваме вентилатора.
    outputs["fan"] = True # Включваме вентилатора.

```

-
-
-
-
-

• [examples/31_case_study/01_bathroom_control.py](#)

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.

2.

3.

Ако не работи

-
-

Нататък

Глава 35. Декомпозиция на две FSM

```
lamp_state = "OFF" # Това е текущото състояние на FSM за лампата.  
fan_state = "OFF" # Това е текущото състояние на FSM за вентилатора.  
  
event_sequence = [] # Започваме с празен списък.  
if button.value() == 0: # Проверяваме дали SW1 е натиснат в момента.  
    event_sequence.append("BUTTON") # Добавяме реално събитие от бутона.  
event_sequence.extend(["HUMIDITY_HIGH", "TIMEOUT_LAMP", "TIMEOUT_FAN"]) # Добавяме симулирани събития.  
  
for event_name in event_sequence: # Обработваме събитията последователно и за двете машини.  
    if event_name == "BUTTON": # Бутонът управлява лампата.  
        lamp_state = "ON" if lamp_state == "OFF" else "OFF" # Превключваме FSM на лампата.  
    if event_name == "HUMIDITY_HIGH": # Високата влажност влияе на FSM на вентилатора.  
        fan_state = "ON" # Включваме вентилатора.
```

-
-
-

- `examples/31_case_study/02_two_fsms.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

-
-

Нататък

Част X: UART комуникация

Глава 36. Базова UART комуникация

```
from machine import UART # Импортираме UART класа за серийна комуникация.
import time # Импортираме time за кратко закъснение.

uart = UART(9, 115200) # Създаваме UART(9) на 115200 baud (TX=P602, RX=P601).
message = "Здравей от VK_RA4M2!\r\n" # Подготвяме съобщение за изпращане.
bytes_sent = uart.write(message) # Изпращаме съобщението по TX пина.

time.sleep_ms(100) # Кратко закъснение, за да пристигнат данни (ако има loopback).
available = uart.any() # Проверяваме дали има налични байтове за четене.
if available > 0: # Ако има данни за четене:
    received = uart.read(available) # Четем наличните байтове от RX буфера.
```



```
print("Получено:", received) # Показваме получените данни.
```

- UART(0) TX=P411 RX=P410
- UART(2) TX=P302 RX=P301
- UART(7) TX=P401 RX=P402 CTS=P403
- UART(9) TX=P602 RX=P601 CTS=P603

-
-
-

- examples/32_uart/01_uart_basic.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
3. any() read()

Упражнения

- 1.
- 2.
3. \r\n

Ако не работи

-
-

Нататък

Част XI: Външни модули по интерфейси

Глава 37. Работа с външни модули по интерфейси

I2C SPI UART

1.

2. I2C SPI UART

3. CS RST INT BUSY EN

4.

VK_RA4M2

- I2C(0) P400 SW1
- I2C(1) P100/P101 hardware SPI
- UART(9)

- I2C
- SPI CS
- UART

1. GND

2.

3.

4.

5.

6.

I2C

```
from machine import I2C # Импортираме I2C за базов probe на външна шина.
```

```
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на безопасни 100 kHz.
```

```
print(i2c.scan()) # Сканираме шината за адреси на свързани I2C модули.
```

SPI

```

from machine import SPI, Pin # Импортираме SPI и Pin за frame-базиран обмен с външен м
одул.

cs = Pin("P103", Pin.OUT, value=1) # Държим CS неактивен преди началото на транзакцият
а.

spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P
101"), miso=Pin("P100"), cs=cs) # Създаваме SPI с консервативни настройки.

cs.value(0) # Активираме target модула чрез CS.

spi.write_readinto(b"\x9F\x00\x00\x00", bytearray(4)) # Изпращаме команда и четем едно
временно отговора.

cs.value(1) # Освобождаваме модула след края на frame-a.

```

UART

```

from machine import UART # Импортираме UART за сериен обмен с външен модул.

uart = UART(9, 115200) # Създаваме UART(9) с често срещана начална скорост.

uart.write(b"AT\r\n") # Изпращаме примерна текстова команда към модула.

print(uart.read()) # Четем каквото е пристигнало към този момент.

```

- I2C scan()
- SPI CS
- UART baud rate TX/RX

datasheet

- I2C
- SPI polarity phase CS
- UART baud rate

- examples/34_external_modules/01_i2c_module_probe.py
- examples/34_external_modules/02_spi_module_transaction.py
- examples/34_external_modules/03_uart_module_request_response.py

Резюме

- I2C SPI UART
-
- VK_RA4M2

Самопроверка

- 1.
2. SPI I2C
3. UART(9) I2C(1) SPI

Упражнения

1. power pins
protocol probe
2. 01_i2c_module_probe.py
3. 03_uart_module_request_response.py

Ако не работи

- GND
- baud rate SPI mode
- RST CS BUSY INT

Нататък

Част XII: asyncio като учебна секция

Глава 38. Основи на asyncio

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.

async def blink_task(period_ms): # Асинхронна задача за мигане с зададен период.
    while True: # Безкрайно мигане.
        led.value(LED_ON) # Светваме LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението за period_ms.
```

```
led.value(LED_OFF) # Гасим LED1.  
await asyncio.sleep_ms(period_ms) # Отстъпваме управлението отново.
```

- `create_task`
- `asyncio.gather`
-

- `examples/33_asyncio/01_asyncio_basics.py`

Резюме

- `asyncio`
-
-

Самопроверка

1. `create_task`
2. `asyncio`
- 3.

Упражнения

- 1.
- 2.
- 3.

Ако не работи

- `await`
-

Нататък

Глава 39. `asyncio` Event синхронизация

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.  
from machine import Pin # Импортираме Pin за реален хардуерен достъп.
```

```

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.
button_event = asyncio.Event() # Създаваме Event обект за синхронизация между задачи.

async def button_producer(): # Тази задача следи SW1 и сигнализира при натискане.
    last_value = 1 # Последна стойност на бутона (ненатиснат).
    current_value = button.value() # Четем SW1.
    if current_value == 0 and last_value == 1: # Нов натиск (falling edge).
        button_event.set() # Сигнализираме Event-а.

async def led_consumer(): # Тази задача чака Event и мига LED1.
    await button_event.wait() # Чакаме Event-а (блокираме кооперативно).
    button_event.clear() # Нулираме Event-а за следващо използване.

```

- `asyncio.Event`

-
-

- `examples/33_asyncio/02_asyncio_event_flag.py`

Резюме

- `asyncio.Event`
-
-

Самопроверка

1. `asyncio.Event`
- 2.
- 3.

Упражнения

- 1.
- 2.
3. `set()`

Ако не работи

- `clear()`

-

Нататък

Част XIII: Специализирани светлинни модули

Глава 40. NeoPixel

NeoPixel

neopixel

- P500
- P112 DIN
- 1

- P500
-
-

NeoPixel

- 1.
- 2.
3. write()

```
from machine import Pin # Импортираме Pin за power-enable и data линията.
```

```
import neopixel # Импортираме neopixel драйвера за RGB модула.
```

```
vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулет не е активен в този setup.
```

```
np = neopixel.NeoPixel(Pin("P112"), 1) # Създаваме NeoPixel обект за един пиксел върху P112.
```

```
np[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
```

```
np.write() # Изпращаме буфера и реално сменяме цвета на модула.
```

- `(R, G, B)`
- `write()`

40

NeoPixel

- DIN P112
- P500
-
-

- `examples/neopixel.py`
- `examples/neopixel_test.py`

Резюме

- NeoPixel
- `power` `write()`
-

Самопроверка

1. NeoPixel
2. `write()` `np[0] = (...)`
3. P500

Упражнения

1. 2 4
- 2.
3. SW1

Ако не работи

- P500
- DIN
-

•

P500

Нататък

WS2812 machine.WS2812

Глава 41. WS2812 през machine.WS2812 и SCI TX-only

WS2812B

neopixel machine.bitstream()
machine.WS2812

SCI
TXD/MOSI

•

DIN

•

SCK MISO

•

write() NeoPixel

SCI

SCI

3-bit

•

SCI 2.857143 MHz

• 5-bit

•

LOW

•

LOW latch/reset

•

DTC IRQ

•

•

write()

•

baudrate

- SCI0 P101 P205 P411
- SCI1 P213 P401 P501
- SCI2 P102 P112 P302
- SCI9 P109 P203 P409 P602

P112

SCI2

- machine.WS2812
- UART(2)
- SPI(2) SCI2

SCI

```
from machine import Pin # Импортираме Pin за захранването и data линията.
from machine import WS2812 # Импортираме специализирания драйвер за WS2812.
import time # Импортираме time за кратко изчакване след включване на захранването.

vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулет не е активен в този setup.

time.sleep_ms(100) # Изчакваме power-enable линията и модулет да се стабилизира.

strip = WS2812(pixel_count=1, pin=Pin("P109"), channels=3) # Създаваме драйверен обект върху алтернативния pin P109.

strip[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.

strip.write() # Изпращаме буфера към модула през SCI TX-only backend-а.
```

NeoPixel

write()

- WS2812
- SCI TX/MOSI
-
- P500

- P500
-

1. pixel_count=1
2. red green blue white off
- 3.

VK_RA4M2

machine.WS2812

- examples/ws2812_sci.py
- examples/ws2812_sci_test.py
- examples/ws2812_sci_chase_145.py
- examples/ws2812_sci_p101.py
- examples/ws2812_sci_p109.py

Резюме

- WS2812 machine
SCI TX/MOSI
-

- WS2812 UART SCI
SPI

Самопроверка

1. neopixel.NeoPixel machine.WS2812
2. DIN -> избрания TXD/MOSI pin
SCI
3. P109 UART(9) machine.WS2812

Упражнения

1. P112 P109
2. pixel_count 2 4
3. SCI0 SCI1
4. examples/ws2812_sci_chase_145.py 1 + 12*12
P500

Ако не работи

- `machine.P500`
`DIN`
- `SCI`
`UART` `SPI`
- `P500`
- `P500`
- `neopixel`
`SCI TX/MOSI`

Нататък

`neopixel` `machine.WS2812`

Каталог на всички примери

- `examples/01_blink_async.py` `LED1`
- `examples/02_two_tasks.py` `asyncio`
- `examples/03_pwm_ab_same_channel.py`
- `examples/04_touchpad_async.py`
- `examples/neopixel.py` `P500` `P112`
- `examples/neopixel_test.py`
- `examples/ws2812_sci.py` `machine.WS2812` `SCI2` `P112`
- `examples/ws2812_sci_test.py` `machine.WS2812`
- `examples/ws2812_sci_chase_145.py` `1 + 12*12`
- `examples/ws2812_sci_p101.py` `machine.WS2812`
`SCI0` `P101`
- `examples/ws2812_sci_p109.py` `machine.WS2812`
`SCI9` `P109`
- `examples/20_language_basics/01_sample_program.py`
- `examples/20_language_basics/02_variables_and_constants.py`
- `examples/20_language_basics/03_operators_conditions_loops.py`
- `examples/20_language_basics/04_enums.py` `const()`
- `examples/20_language_basics/05_arrays.py` `array` `bytearray`
- `examples/20_language_basics/06_structures.py` `dict` `namedtuple`

- examples/20_language_basics/07_pointers_equivalents.py memoryview
machine.mem32
- examples/20_language_basics/08_functions.py
- examples/21_digital_io/01_digital_output.py LED1
- examples/21_digital_io/02_digital_input.py SW1
- examples/22_analog/01_analog_input_adc.py
- examples/22_analog/02_pwm_output.py
- examples/23_timing/01_delays_and_timers.py
uasyncio
- examples/23_timing/02_interrupts.py
- examples/23_timing/03_button_debounce.py uasyncio
- examples/23_timing/04_software_timer_minus1.py Timer(-1)
- examples/23_timing/05_timer_compare_hw_sw.py Timer(1) Timer(-1)
- examples/23_timing/06_rtc_basic.py
- examples/23_timing/07_adc_timing_measure.py ADC.read()
time.ticks_us()
- examples/23_timing/08_interrupt_latency_measure.py
- examples/23_timing/09_button_polling_events.py polling PRESS
RELEASE CLICK LONG_PRESS
- examples/23_timing/10_button_irq_deferred.py
- examples/23_timing/11_button_soft_timer_hold.py Timer(-1) PRESS RELEASE
HOLD_START HOLD_REPEAT
- examples/23_timing/12_button_shift_debounce.py
- examples/23_timing/13_hardware_timer_periodic_oneshot.py Timer.PERIODIC
Timer.ONE_SHOT period() counter()
- examples/23_timing/14_hardware_timer_output_compare.py output compare
channel(1) channel(2)
- examples/23_timing/15_hardware_timer_input_capture.py input capture
pulse width
- examples/23_timing/16_hardware_timer_event_count.py event count N
- examples/23_timing/17_fast_irq_two_buttons_queue.py fast ASM IRQ
- examples/AGT_TEST_CHECKLIST.md
- examples/24_i2c/01_i2c_master_basic.py
- examples/24_i2c/02_i2c_runtime_pins.py

- `examples/24_i2c/03_softi2c_basic.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/25_spi/01_spi_basic.py`
- `examples/25_spi/02_softspi_basic.py`
- `examples/26_touchpad/01_touchpad_basic.py`
- `examples/26_touchpad/02_touchpad_diagnostics.py`
- `examples/27_storage/01_flash_blockdev.py` `renesas.Flash` `/flash`
- `examples/27_storage/02_dataflash_basic.py` `dataflash`
- `examples/27_storage/03_dataflash_state_led.py` `dataflash`
- `examples/28_machine_misc/01_machine_info_and_id.py` `machine.info` `freq` `unique_id`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py` `disable_irq` `enable_irq` `mem32` `bootloader`
- `examples/28_machine_misc/03_sleep_modes.py` `sleep` `lightsleep` `deepsleep`
- `examples/28_machine_misc/04_memory_management.py` `gc` `micropython.mem_info` `bytearray` `memoryview`
- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`
- `examples/30_fsm/01_fsm_lamp_button.py`
- `examples/30_fsm/02_fsm_table.py`
- `examples/30_fsm/03_fsm_timeout.py`
- `examples/30_fsm/04_conditions_vs_events.py`
- `examples/30_fsm/05_guard_functions.py`
- `examples/30_fsm/06_priorities.py`
- `examples/30_fsm/07_event_queue.py`
- `examples/30_fsm/08_tickless_sleep.py`
- `examples/31_case_study/01_bathroom_control.py`
- `examples/31_case_study/02_two_fsms.py`
- `examples/32_uart/01_uart_basic.py`
- `examples/34_external_modules/01_i2c_module_probe.py`
- `examples/34_external_modules/02_spi_module_transaction.py` `CS`
- `examples/34_external_modules/03_uart_module_request_response.py`
- `examples/33_asyncio/01_asyncio_basics.py`
- `examples/33_asyncio/02_asyncio_event_flag.py`

- `examples/index.py`

Какво още не е потвърдено на реален борд

VK_RA4M2

-
-
-
- P500/P112
- machine.WS2812 SCI TX/MOSI
-
- Flash dataflash
- lightsleep deepsleep

Заклучение

-
- 67
-
- VK_RA4M2